Chapter 4 — Memory Hierarchy and HBM Deep Dive

> *"GPUs don't run out of compute. They run out of memory bandwidth. Every performance optimization in AI infrastructure is, at its core, a battle against the memory wall."* — Principal Performance Architect's First Principle

---

Chapter Overview

This chapter answers one deceptively simple question: *Why is moving data harder than computing with it?* The answer determines whether a kernel runs at 5% or 95% hardware efficiency, whether an inference system serves 10 users or 10,000 users, and whether your cluster achieves 40% or 80% MFU.

Sections in this chapter:

- 4.1 The Memory Wall — Why It Exists and Won't Go Away
- 4.2 The Five-Tier GPU Memory Hierarchy
- 4.3 HBM Architecture — Physical Construction and Die Stacking
- 4.4 HBM Generations — HBM2 to HBM4
- 4.5 Bandwidth Utilization — Access Patterns and Their Costs
- 4.6 The Roofline Model Applied — Finding Your Regime
- 4.7 Memory Bandwidth Optimization Techniques
- 4.8 FlashAttention — A Case Study in Memory Hierarchy Mastery
- 4.9 KV Cache Memory Engineering
- 4.10 Profiling Memory Bottlenecks
- 4.11 The Memory Performance Decision Tree
- 4.12 Key Takeaways and Review Questions

4.1 The Memory Wall — Why It Exists and Won't Go Away

4.1.1 The Fundamental Imbalance

Semiconductor physics has produced a persistent and widening gap between compute throughput and memory bandwidth. This gap — the *memory wall* — is the defining constraint of modern AI infrastructure.

```
GPU COMPUTE vs. HBM BANDWIDTH SCALING (2018-2025):

  Year   GPU     Peak BF16 TFLOPS   HBM Bandwidth   Compute:BW Ratio
  ──────────────────────────────────────────────────────────────────
  2018   V100    112 TFLOPS         900 GB/s         124 FLOP/byte
  2020   A100    312 TFLOPS         2.0 TB/s         156 FLOP/byte
  2022   H100    989 TFLOPS         3.35 TB/s        295 FLOP/byte
  2024   B200    ~2,250 TFLOPS      8.0 TB/s         281 FLOP/byte [SHIPPED]
  2025   B300    ~2,800 TFLOPS      12.0 TB/s        233 FLOP/byte [ANNOUNCED]
  ──────────────────────────────────────────────────────────────────


  KEY OBSERVATION:
    Compute grew ~20× over 6 years.
    Bandwidth grew only ~13× over the same period.
    The compute:bandwidth RATIO widened from 124 → 295 FLOP/byte.

  WHAT THIS MEANS:
    Operations that were compute-bound on A100 may be memory-bound on H100.
    The ridge point keeps rising. Every GPU generation reclassifies workloads.
    This is why you must re-profile on new hardware — never assume regime is
stable.
```

4.1.2 Why Bandwidth Is Hard to Scale

Memory bandwidth is limited by three physical realities that resist the same scaling laws that drive compute density improvements:

Physical Reality 1 — Pin count and package I/O: Every data pin on a chip package can carry approximately 64 Gbps in modern standards. Adding bandwidth means adding more pins, which increases package size, cost, and thermal footprint non-linearly. GDDR6X gaming GPU: 384-bit bus total. HBM3 H100: 5,120-bit bus total (5 stacks × 1024-bit). HBM achieves this by going *vertical* instead of horizontal.

Physical Reality 2 — DRAM cell physics: DRAM stores charge in capacitors that must be periodically refreshed. Refresh overhead:

- Room temperature: 3–5% of total bandwidth consumed by refresh cycles
- AI datacenter temperature (45°C+): 8–12% overhead

This is why peak HBM bandwidth is a ceiling, not a floor — sustained utilization tops out at ~88–92% of the spec number regardless of access pattern quality.

Physical Reality 3 — Power density: Driving a memory bus at high frequency and wide width consumes power unavailable for compute. The H100's HBM3 interface consumes approximately

40–60W just for the memory subsystem — before any Tensor Core work. HBM3e's lower operating voltage (1.05V vs. HBM3's 1.1V) reduces this overhead, freeing thermal budget for compute.

4.2 The Five-Tier GPU Memory Hierarchy

A complete GPU system has five distinct memory tiers. **Every performance optimization is fundamentally about moving data to the fastest tier that can hold it and reusing it as many times as possible before it leaves that tier.**

```
GPU MEMORY HIERARCHY — H100 SXM5: [SHIPPED]

  Tier  Level                Capacity      Bandwidth      Latency      Scope
  ──────────────────────────────────────────────────────────────────────────
  1     Registers            256 KB/SM     ~100 TB/s      1 cycle      Per thread
  2     L1 / Shared Memory   228 KB/SM     ~80 TB/s       20-30 cycles Per SM
block
  3     L2 Cache             50 MB total   ~12 TB/s       150 cycles   All SMs
  4     HBM3 (GPU DRAM)      80 GB         3.35 TB/s      400+ cycles  All SMs
  5     PCIe 5.0 (CPU-GPU)   —             ~128 GB/s      ~1 μs        Host access
  ──────────────────────────────────────────────────────────────────────────

  SUPPLEMENTARY INTERCONNECTS (not in hierarchy, but in the system):
    NVLink 4.0 (GPU-GPU):     900 GB/s   (intra-node, peer GPU access)
    InfiniBand NDR (node-node): ~50 GB/s  (inter-node, collective ops)

  BANDWIDTH RATIOS (what the gaps mean in practice):
    L1  vs Registers:  80 TB/s  / 100 TB/s  = 0.8× (nearly same tier)
    L2  vs L1:         12 TB/s  / 80 TB/s   = 6.7× gap
    HBM vs L2:         3.35 TB/s / 12 TB/s  = 3.6× gap
    PCIe vs HBM:       128 GB/s / 3,350 GB/s = 26× gap
    IB   vs PCIe:      50 GB/s  / 128 GB/s  = 2.6× gap

  THE ENGINEERING PRINCIPLE:
    Every tier boundary crossed costs bandwidth.
    FlashAttention: keeps attention scores in SMEM, not HBM (tier 2 vs tier 4).
    Operator fusion: keeps activations in registers, not HBM (tier 1 vs tier 4).
    KV cache: managed in HBM — the entire serving optimization problem is about
              making HBM access as efficient as possible (tier 4).
    Gradient checkpointing: trades HBM reads (recompute) against HBM storage.
```

4.2.1 Shared Memory — The Programmer-Controlled Cache

Shared memory (SMEM) is physically the same SRAM as L1 cache on H100, but partitioned under programmer control. This is the primary kernel optimization surface.

```
SMEM TILING — HOW IT CHANGES ARITHMETIC INTENSITY:

  GEMM goal: compute C = A × B for large matrices A, B, C.

  WITHOUT SMEM TILING (naive):
    Load A[i,k] from HBM for every C[i,j] computation.
    Each element of A is loaded S×N times (once per output row).
    HBM traffic ~ M×K×N reads for A alone.
    Arithmetic Intensity ≈ 2 FLOP/byte  -> deeply memory-bound

  WITH SMEM TILING (cuBLAS / CUTLASS approach):
    Load one tile of A [tile_M × tile_K] into SMEM.
    Load one tile of B [tile_K × tile_N] into SMEM.
    Compute all C outputs that use these tiles (multiple operations, one load).
    Each element loaded from HBM ONCE per tile, reused tile_size times in SMEM.

  Arithmetic Intensity with tiling:
    AI = (2 × tile_M × tile_K × tile_N) / ((tile_M × tile_K + tile_K × tile_N) ×
2B)
       = 2 × tile_size   [FLOP/byte]  (simplified for square tiles)

  SMEM tile sizing for H100:
    Available SMEM: up to 228 KB
    At BF16 (2 bytes/element): 228,000 / 2 = 114,000 elements max
    Square tile: sqrt(114,000) ≈ 337 elements per side
    Practical tiles: 128×256 or 256×128 (HW-aligned for wgmma)

  Arithmetic Intensity with 128-element tile: 2 × 128 = 256 FLOP/byte
  Arithmetic Intensity with 256-element tile: 2 × 256 = 512 FLOP/byte

  H100 BF16 ridge point: ~295 FLOP/byte
  -> 128-element tile: NEAR ridge (near-compute-bound) ✓
  -> 256-element tile: ABOVE ridge (compute-bound)     ✓✓
  -> No tiling (AI~2): FAR below ridge (deeply memory-bound) ×
```

4.3 HBM Architecture — Physical Construction and Die Stacking

High Bandwidth Memory achieves its extraordinary bandwidth by abandoning the traditional PCB-trace approach entirely. Understanding the physical construction explains both the bandwidth numbers and why capacity scales slower than bandwidth.

4.3.1 Physical Construction

```
HBM3 STACK ARCHITECTURE (H100 SXM5):

  Traditional GDDR6X (gaming GPU):
    DRAM chips on PCB, connected via 384-bit bus
    Bus length: ~2-5 cm traces on PCB
    Voltage: 1.35V
    Bandwidth: ~1 TB/s total

  HBM3 (AI GPU):
    DRAM dies stacked VERTICALLY on an interposer
    GPU die sits on the SAME silicon interposer as HBM stacks
    Connected via Through-Silicon Vias (TSVs): 55 µm pitch, thousands per stack
    Bus width: 1,024 bits per stack (vs 384-bit total for GDDR6X)
    Voltage: 1.1V (HBM3) / 1.05V (HBM3e) — lower power at higher BW

  H100 PACKAGE LAYOUT (CoWoS: Chip-on-Wafer-on-Substrate):

     ┌──────────────────────────────────────────────────────┐
     │              Silicon Interposer                      │
     │  ┌──────────────┐   ┌────┐  ┌────┐  ┌────┐  ┌────┐ │
     │  │              │   │HBM│  │HBM│  │HBM│  │HBM│ │
     │  │  GH100 GPU   │   │ 1 │  │ 2 │  │ 3 │  │ 4 │ │
     │  │  (compute)   │   │   │  │   │  │   │  │   │ │
     │  │              │   │   │  │HBM│  │   │  │   │ │
     │  │              │   │   │  │ 5 │  │   │  │   │ │
     │  └──────────────┘   └────┘  └────┘  └────┘  └────┘ │
     └──────────────────────────────────────────────────────┘

    H100: GPU die + 5× HBM3 stacks = 80 GB total
    H200: GPU die + 6× HBM3e stacks = 141 GB total  (same GPU die!)

  ONE HBM3 STACK (H100 variant):
     ┌──────────────────────────┐
     | DRAM Die #5 — 16 GB      |  ← Top die
     ├──────────────────────────┤
     | DRAM Die #4 — 16 GB      |
```

```
    ├──────────────────────────┤
    | DRAM Die #3 — 16 GB      |
    ├──────────────────────────┤
    | DRAM Die #2 — 16 GB      |
    ├──────────────────────────┤
    | DRAM Die #1 — 0 GB       |  ← Base logic die (signal routing, PHY)
    └──────────────────────────┘

       ↕ TSVs (1024 data bits + control)
    GPU die / interposer

    Effective per-stack: 4 × 16 GB = 64 GB capacity
    Wait — H100 has 80 GB on 5 stacks = 16 GB per stack (different stacking
config)
    H200 HBM3e: 141 GB on 6 stacks = ~23.5 GB per stack (24 GB/stack spec)

HBM BANDWIDTH DERIVATION:
  BW = stacks × (bus_width_bits / 8) × clock_freq × 2  (DDR: 2 transfers/cycle)

  H100 HBM3:
    BW = 5 × (1024/8) × 819 MHz × 2
       = 5 × 128 bytes × 819,000,000 × 2
       = 5 × 209,664,000,000 bytes/s
       = 1,048,320,000,000 bytes/s × 3.2 (effective rate)  ← actual HBM3 spec
       = 3.35 TB/s  [SHIPPED] ✓

  H200 HBM3e:
    BW = 6 stacks × ~800 GB/s per stack = 4.8 TB/s  [SHIPPED]
    (HBM3e achieves higher data rate per stack than HBM3 at same voltage)
```

4.3.2 Why HBM Capacity Scales Slowly

```
THE HBM CAPACITY SCALING CONSTRAINT:

  Why not just add more stacks?

  1. INTERPOSER AREA LIMIT:
     H100 package is ~814 mm² — near the EUV reticle limit (~858 mm²).
     Each HBM stack adds ~100 mm² of interposer area.
     Adding 2 more stacks would require a larger interposer, reducing yield
     non-linearly (defect probability grows with area).

  2. TSV YIELD:
     Each stack has ~1000+ TSVs at 55 μm pitch.
```

```
     At 0.01% defect rate: 10 defective TSVs per 100K TSVs.
     Taller stacks (more DRAM dies) multiply the probability of one bad TSV.
     Bad TSV = defective stack = reduced yield.

  3. THERMAL EXTRACTION:
     Each HBM stack dissipates 5-10W internally.
     Heat must travel through stacked dies via TSVs to the interposer surface.
     Thermal resistance of stacked dies is non-trivial.
     HBM die temperature typically runs 15-25°C hotter than GPU die under load.
     More stacks = more internal heating = more thermal management complexity.

  SCALING LAW COMPARISON:
    Compute: grows with process node shrink (4nm → 3nm) → roughly 2× per 2 years
    HBM bandwidth: grows with data rate improvements → ~2× per 2 years
    HBM capacity: grows with die density and stack height → ~1.5× per 2 years

  PRACTICAL CONSEQUENCE:
    H100 → B200: compute 4.5× more, BW 2.4× more, capacity 2.4× more
    "HBM4 era" (B300): estimated 3× more BW vs B200, ~50% more capacity

    Bandwidth grows faster than capacity.
    This means bandwidth-per-GB improves every generation.
    Quantization (fewer bytes per weight) becomes MORE impactful each generation
    because each generation's HBM can sustain higher rates per byte accessed.
```

4.4 HBM Generations — HBM2 to HBM4

```
HBM GENERATION COMPARISON TABLE:

  Gen     GPU (example) BW/stack   Total BW   Cap/stack  Voltage  Rate
  ──────────────────────────────────────────────────────────────────────
  HBM2    V100          256 GB/s   900 GB/s   16 GB      1.2V     2 Gbps
[SHIPPED]
  HBM2e   A100 SXM4     307 GB/s   2.0 TB/s   16 GB      1.2V     3.2
Gbps[SHIPPED]
  HBM3    H100 SXM5     819 GB/s   3.35 TB/s  16 GB      1.1V     6.4
Gbps[SHIPPED]
  HBM3e   H200 SXM5     ~1.2 TB/s  4.8 TB/s   24 GB      1.05V    9.6
Gbps[SHIPPED]
  HBM3e   B200 SXM      ~1.6 TB/s  8.0 TB/s   24 GB      1.05V    9.6
Gbps[SHIPPED]
```

```
  HBM4    B300/Rubin    ~2.0 TB/s  ~12.0 TB/s 32 GB      ~1.0V    ~16
Gbps[ANNOUNCED]
  ──────────────────────────────────────────────────────────────────────

  KEY TRENDS:
    Bandwidth grew 13× from HBM2 → HBM4 (estimated)
    Capacity grew 2× per stack from HBM2 → HBM4
    Voltage dropped from 1.2V → ~1.0V (power efficiency improved)
    Data rate grew 8× (electrical improvement, not just more stacks)

GENERATION-TO-GENERATION DECODE THROUGHPUT IMPACT:

  For LLaMA-3 70B BF16 (140 GB model, batch=1, decode-limited):
    Theoretical max decode = HBM_bandwidth / model_bytes × 0.70 (practical)

  GPU     HBM BW    Theoretical max   Practical (70%)  GPUs needed
  ──────────────────────────────────────────────────────────────────
  A100    2.0 TB/s  14.3 tok/s×2GPU  ~20 tok/s (TP=2) 2 GPUs
  H100    3.35 TB/s 23.9 tok/s×2GPU  ~33 tok/s (TP=2) 2 GPUs
  H200    4.8 TB/s  34.3 tok/s×1GPU  ~24 tok/s (1 GPU) 1 GPU ← fits!
  MI300X  5.3 TB/s  37.9 tok/s×1GPU  ~27 tok/s (1 GPU) 1 GPU ← fits!
  B200    8.0 TB/s  57.1 tok/s×1GPU  ~40 tok/s (1 GPU) 1 GPU ← fits!
  ──────────────────────────────────────────────────────────────────

  THE H200 LESSON:
    Same compute die as H100 (identical SM count, identical Tensor Cores).
    4.8 TB/s vs 3.35 TB/s = +43% bandwidth.
    Single-GPU 70B serving: eliminates TP=2 requirement (saves 1 GPU, eliminates
    AllReduce communication overhead).
    Result: ~40% throughput improvement for 70B inference with ZERO compute
change.
    This is the purest demonstration that for LLM decode: bandwidth > TFLOPS.
```

4.5 Bandwidth Utilization — Access Patterns and Their Costs

Peak HBM bandwidth is a theoretical ceiling. Real workloads achieve 60–92% of peak depending on access pattern. The difference between a well-written and a poorly-written kernel on the same hardware can be 13×.

```
HBM3 BANDWIDTH UTILIZATION BY ACCESS PATTERN (H100, 3.35 TB/s peak):

  Access Pattern                  Achieved BW     Utilization    Notes
  ──────────────────────────────────────────────────────────────────────
  Sequential streaming (memcpy)   3.0-3.1 TB/s    90-92%        Ideal case
  GEMM weight loading (large)     2.7-2.9 TB/s    80-87%        Well-tiled CUTLASS
  GEMV (decode, large batch)      2.2-2.6 TB/s    65-78%        Varies with shape
  Scattered reads (gather ops)    0.8-1.5 TB/s    24-45%        Non-contiguous
access
  Random access (hash lookups)    0.3-0.8 TB/s    9-24%         No locality
  Uncoalesced warp access         0.1-0.4 TB/s    3-12%         Worst case
  ──────────────────────────────────────────────────────────────────────

  The difference between sequential and uncoalesced: 25-30×.
  An uncoalesced kernel on H100 achieves less HBM bandwidth than a well-written
  kernel on A100. Hardware generation is irrelevant if access patterns are broken.
```

4.5.1 Memory Coalescing — The Single Most Important Access Pattern Rule

```
COALESCING: What It Is and Why It Matters

  All 32 threads in a warp execute simultaneously.
  When all threads load memory, the GPU issues memory transactions.

  COALESCED ACCESS (all threads access consecutive addresses):
    Thread 0 loads address 0x1000
    Thread 1 loads address 0x1002  (BF16 = 2 bytes apart)
    Thread 2 loads address 0x1004
    ...
    Thread 31 loads address 0x103E

    -> All 32 loads fit in ONE 128-byte memory transaction.
    -> 32 elements loaded for the cost of 1 transaction.
    -> Bandwidth utilization: ~90%

  UNCOALESCED ACCESS (threads access scattered addresses):
    Thread 0 loads address 0x1000
    Thread 1 loads address 0x2000  (4096 bytes apart)
    Thread 2 loads address 0x3000
    ...

    -> Each load is a SEPARATE memory transaction (32 total).
```

```
    -> 32 elements loaded for the cost of 32 transactions.
    -> Only 2 bytes of data per 128-byte transaction (1.6% efficiency).
    -> Bandwidth utilization: ~3-12%

MATRIX ACCESS PATTERN EXAMPLE:
  Matrix A [N×M], BF16, stored row-major:

  Row-major storage: A[0][0], A[0][1], A[0][2], ..., A[0][M-1], A[1][0], ...

  Accessing A[row][tid] (all threads same row, consecutive columns):
    -> COALESCED. All threads hit consecutive memory. One transaction.

  Accessing A[tid][col] (all threads different rows, same column):
    -> UNCOALESCED. Stride = M elements = M×2 bytes apart.
    -> 32 separate transactions. 13× slower for same data volume.

  FIX: Transpose the matrix, OR load to SMEM with transposed layout:
    Load one tile of A into SMEM with transposed access.
    Compute using SMEM (always fast, bank-conflict-free with proper padding).
    Only HBM access is the initial load — done coalesced.

DIAGNOSTIC:
  Nsight Compute: l2_global_load_bytes vs. l1tex_t_bytes_pipe_lsu_mem_global_op_ld
  Ratio should be close to 1.0. If >> 1.0: coalescing inefficiency.
  Also: sm__sass_l1tex_t_sectors_pipe_lsu_mem_global_op_ld.sum — high sector
  count relative to requests = poor coalescing.
```

4.6 The Roofline Model Applied — Finding Your Regime

Chapter 1 introduced the roofline. This section applies it at the granularity of individual LLM operations so you can predict regime before profiling.

```
ROOFLINE — H100 SXM5 BF16 (Dense): [SHIPPED]
  Peak TFLOPS:    989 TFLOPS
  HBM bandwidth:  3.35 TB/s
  Ridge point:    ~295 FLOP/byte

OPERATION CLASSIFICATION TABLE — LLaMA-3 70B:

  Operation                 Matrix Shape            AI (FLOP/byte)  Regime
  ──────────────────────────────────────────────────────────────────────────
  Embedding lookup          [V × d_model]           ~1              Mem-bound
  RMSNorm (per token)       [d_model]               ~3-5            Mem-bound
  QKV proj (decode, B=1)    [1 × d_model]×[d×3d]   ~1              Mem-bound
  QKV proj (decode, B=32)   [32 × d_model]×[d×3d]  ~32             Mem-bound
  Decode attention (B=1)    [1 × C × d_head]        ~1              Mem-bound
  FFN down-proj (B=1)       [1 × d_ff]×[d_ff × d]  ~1              Mem-bound
  FFN down-proj (B=128)     [128 × d_ff]×[d_ff×d]  ~128            Mem-bound
  Prefill GEMM (B=64,S=256) [16K × d_model]         ~200            Near ridge
  FlashAttention (S=2K)     tiled                   ~200-250        Near ridge
  Training GEMM (large)     [d × d_ff] large        ~300-500        Compute-bound
  ──────────────────────────────────────────────────────────────────────────

  CRITICAL INSIGHT:
    Every operation at decode time (batch=1 to ~128) is memory-bandwidth-bound.
    Only large-batch training GEMMs are compute-bound.
    The AI/ML inference engineer's primary job is managing HBM bandwidth.
    Tensor Core efficiency is a training concern; bandwidth efficiency is an
    inference concern.

WHEN DOES DECODE BECOME COMPUTE-BOUND?

  For LLaMA-3 70B BF16, the FFN GEMM:
    AI = (2 × B × d_model × d_ff) / (d_model × d_ff × 2B)
       = 2 × batch_size / 2
       = batch_size FLOP/byte

  To reach H100 ridge point (295 FLOP/byte):
    batch_size = 295  -> need ~295 concurrent tokens

  In practice: serves need ~256-512 batch size to become compute-bound.
  Continuous batching (Chapter 6) exists specifically to reach this threshold.
```

4.7 Memory Bandwidth Optimization Techniques

4.7.1 Operator Fusion — Eliminating HBM Round-Trips

Fusion is the single highest-impact memory optimization available without changing algorithms. By merging multiple kernel launches, intermediate results never touch HBM.

```
UNFUSED EXECUTION: y = RMSNorm(GELU(GEMM(x)))

  Step 1: GEMM     x[HBM] -> GEMM -> z[HBM]     Read: x, Write: z
  Step 2: GELU     z[HBM] -> elem -> g[HBM]     Read: z, Write: g
  Step 3: RMSNorm  g[HBM] -> reduce -> y[HBM]   Read: g, Write: y

  HBM traffic: read_x + 2×z + 2×g + write_y
             = 2 bytes × (S×d) × [1 + 2 + 2 + 1] = 6 × activation_size

  For one FFN sublayer of LLaMA-3 70B (S=4096, d=8192, BF16):
    Activation size = 4096 × 8192 × 2 bytes = 67 MB
    Un-fused HBM traffic: 6 × 67 MB = 402 MB (for intermediate activations alone)
    At 3.35 TB/s HBM: 402 MB / 3350 GB/s = 0.12 ms pure overhead per sublayer

FUSED EXECUTION: single kernel

  Read x from HBM once
  -> GEMM result in SMEM/registers
  -> GELU in registers
  -> RMSNorm in registers (running mean/variance in registers)
  Write y to HBM once

  HBM traffic: read_x + write_y = 2 × 67 MB = 134 MB
  Speedup: 402 / 134 = 3.0× on this memory-bound path

FUSION TARGETS IN LLM INFERENCE (ordered by impact):

  1. FlashAttention (see Section 4.8):
     Fuses QK^T + scale + mask + softmax + AV into ONE kernel.
     Eliminates the S×S attention matrix from HBM entirely.
     Impact: 5-10× for long sequences.

  2. QKV projection fusion:
     Single GEMM: [x] × [W_Q; W_K; W_V] instead of 3 separate GEMMs.
     Saves 2 HBM reads of x. Impact: ~10% per transformer layer.

  3. RMSNorm + GEMM fusion:
     RMSNorm reads activation, GEMM reads it immediately after.
     Fused: activation lives in SMEM/registers between the two.
     Impact: 8% of step time at LLaMA-3 70B scale (see Ch 2, §2.9.4).

  4. Activation function + GEMM:
     SiLU/GELU computed in-register immediately after GEMM output.
```

```
      Standard in all production LLM runtimes (vLLM, TRT-LLM, Megatron).

  5. Residual add + normalization:
      Residual addition is elementwise (one HBM read + write).
      Fused with normalization: one read, one write, both ops in register.
```

4.7.2 Quantization — Reducing Bytes Without Reducing FLOPs

```
QUANTIZATION IMPACT ON DECODE THROUGHPUT (bandwidth-bound workload):

  Model: LLaMA-3 70B, Batch=1, H100 SXM5 (3.35 TB/s)

  Precision    Bytes/param  Model size  Theoretical tok/s   Practical (×0.70)
  ──────────────────────────────────────────────────────────────────────────
  FP32         4 bytes      280 GB      3350/280 = 12        ~8   (impractical)
  BF16/FP16    2 bytes      140 GB      3350/140 = 24        ~17  (baseline)
  FP8 W8A8     1 byte        70 GB      3350/70  = 48        ~34  (2× speedup)
  INT4 W4A16   0.5 bytes     35 GB      3350/35  = 96        ~67  (4× speedup)
  ──────────────────────────────────────────────────────────────────────────


  FUNDAMENTAL RELATIONSHIP (memory-bound decode):
    decode_throughput ∝ 1 / model_bytes_per_token
    = 1 / (N_params × bytes_per_param)

  Halving bytes_per_param → doubles decode throughput. Linear. Direct.
  This relationship holds until:
    (a) Batch size grows and AI crosses the ridge point (compute takes over)
    (b) Dequantization overhead becomes significant (INT2 and below)
    (c) Quality degradation exceeds production tolerance (~1% accuracy drop)

QUANTIZATION SELECTION FRAMEWORK:
  BF16:  Always safe. Use as baseline. No calibration required.
  FP8:   ~0.1-0.3% quality loss. Transformer Engine handles automatically on H100.
         Use whenever compute is available. Free 2× on decode.
  INT8 (W8A8): ~0.5-1% quality loss. Requires per-channel/per-token calibration.
               Good for models where FP8 isn't available (older GPUs).
  INT4 (GPTQ/AWQ): ~1-3% quality loss depending on method.
                   Acceptable for many applications. 4× decode speedup.
  INT2: >5% quality loss on most models. Generally not production-viable.
```

4.7.3 Prefetching and Async Memory Copies

```
ASYNC PREFETCH — HIDING HBM LATENCY:
```

```
  HBM latency: ~400 CPU cycles.
  Without prefetch: CPU issues load, SM stalls 400 cycles waiting for data.
  With prefetch: SM issues load early, continues computing, data arrives ready.

  CUDA async copy (H100+):
    __pipeline_memcpy_async(dst_smem, src_gmem, bytes);
    __pipeline_commit();
    // Do other work here while data is in flight
    __pipeline_wait_prior(0);  // Ensure data has arrived
    // Now use the data

  CUTLASS double-buffering pattern:
    Buffer A: being loaded from HBM while Buffer B is being computed
    Buffer B: being loaded from HBM while Buffer A is being computed
    -> Zero stall time: compute and memory access fully overlapped
    -> Requires 2× SMEM for the tile buffer

  WGMMA async (H100 Hopper-specific):
    The wgmma (warpgroup matrix multiply) instruction is asynchronous.
    SM issues wgmma, immediately issues the next SMEM prefetch.
    Tensor Cores execute while the next tile is arriving in SMEM.
    This is the primary mechanism behind H100's sustained 989 TFLOPS —
    it overlaps compute and data movement at the instruction level.
```

4.8 FlashAttention — A Case Study in Memory Hierarchy Mastery

FlashAttention is the most important memory optimization in the history of LLMs.
It is also the cleanest demonstration of every principle in this chapter applied together.

4.8.1 The Problem: Standard Attention Is Quadratic in HBM Traffic

```
STANDARD ATTENTION HBM TRAFFIC ANALYSIS:

  Inputs: Q, K, V — each [S × d_head] BF16
  Output: O — [S × d_head] BF16
  Temporary: S (attention scores) — [S × S] FP32

  Operation sequence and HBM accesses:
    1. Load Q, K from HBM     : read 2 × S×d × 2B
    2. Compute S = QK^T       :
    3. Write S to HBM         : write S×S × 4B  ← O(S²) WRITE
```

```
    4. Load S from HBM        : read  S×S × 4B  ← O(S²) READ
    5. Apply softmax in-place :
    6. Write softmax(S) to HBM: write S×S × 4B  ← O(S²) WRITE again
    7. Load softmax(S), V     : read  S×S × 4B + S×d × 2B
    8. Compute O = softmax(S)×V:
    9. Write O to HBM         : write S×d × 2B

  TOTAL HBM TRAFFIC ≈ 4 × S² × 4B + 4 × S × d × 2B

  For LLaMA-3 70B, one attention head, S=4096, d_head=128, BF16:
    S² term: 4 × 4096² × 4B = 268 MB per head
    S×d term: 4 × 4096 × 128 × 2B = 4 MB per head
    Total: ~272 MB per head per layer

  LLaMA-3 70B: 64 heads × 80 layers × 272 MB = 1.39 TB HBM traffic
    ...for ONE forward pass through attention alone!

  AT SCALE THIS IS CATASTROPHIC:
    S=8192:  S² = 64× larger → 4× more HBM traffic (S doubled)
    S=32768: S² = 1024× larger → 64× more HBM traffic
    Without FlashAttention, 128K context is physically impossible —
    the attention matrix alone would be 128000² × 4B × 64 heads = 4 TB
    (model itself is only 140 GB).
```

4.8.2 FlashAttention: Tiling to Eliminate O(S²) HBM Traffic

```
FLASHATTENTION ALGORITHM — CORE IDEA:

  Key insight: We don't need to materialize the full S×S attention matrix.
  We can compute the final output O incrementally, one tile at a time,
  if we maintain running statistics (softmax denominator and numerator).

  TILING STRATEGY:
    Divide Q into blocks of size [Br × d]
    Divide K, V into blocks of size [Bc × d]
    SMEM holds: one Q block + one K block + one V block + accumulator

    For each Q block:
      For each K, V block:
        Load Q_block from HBM to SMEM    (once per outer loop)
        Load K_block, V_block from HBM to SMEM
        Compute S_tile = Q_block × K_block^T  in SMEM
        Update running softmax (online normalization trick)
```

```
        Update running output accumulator in SMEM
    Write final output block to HBM once

  FLASHATTENTION HBM TRAFFIC:
    Reads: Q, K, V each once = 3 × S × d × 2B = O(S) not O(S²)
    Writes: O once = S × d × 2B = O(S)
    NO S×S matrix is ever written to or read from HBM.

  TRAFFIC COMPARISON (LLaMA-3 70B, one head, S=4096, d=128, BF16):
    Standard attention: ~272 MB per head
    FlashAttention:     ~6 MB per head (3 × 4096 × 128 × 2B × 2)
    Speedup factor:     ~45× reduction in HBM traffic for attention alone

  THE ONLINE SOFTMAX TRICK — HOW TILED SOFTMAX WORKS:
    Softmax requires knowing the global maximum and sum across ALL keys.
    We cannot compute this without seeing all keys at once — or can we?

    Running statistics:
      max_i = max(max_{i-1}, row_max(S_tile_i))  ← update max as we see tiles
      sum_i = sum_{i-1} × exp(max_{i-1} - max_i) + row_sum(exp(S_tile_i - max_i))
      out_i = out_{i-1} × exp(max_{i-1} - max_i) + exp(S_tile_i - max_i) ×
V_tile_i

    Final normalization: out_final = out_N / sum_N

    This produces EXACTLY the same result as materializing the full S×S matrix
    and applying standard softmax — proven mathematically in Dao et al. (2022).

SMEM REQUIREMENT FOR FLASHATTENTION:
  Tile size Br = Bc must fit in SMEM:
    SMEM needed = Br×d (Q tile) + Bc×d (K tile) + Bc×d (V tile) + Br×Bc (S tile)

  On H100 (228 KB SMEM, BF16):
    228,000 bytes / 2 bytes = 114,000 elements
    Solve: Br×128 + Bc×128 + Bc×128 + Br×Bc = 114,000
    With Br=Bc=64: 64×128 + 64×128 + 64×128 + 64×64 = 28,672 elements
    (well within 114K limit, allowing for larger tiles)

  FlashAttention-3 (H100-specific, 2024):
    Exploits H100 WGMMA async instruction for attention computation.
    Overlaps GEMM (QK^T tile) and softmax computation.
    Achieves ~750 TFLOPS on attention vs ~200 TFLOPS for FA-2.
```

4.9 KV Cache Memory Engineering

The KV cache is the dominant memory consumer in production LLM inference. Every decision about context length, batch size, and GPU count intersects it.

4.9.1 KV Cache Sizing from First Principles

```
KV CACHE MEMORY FORMULA:

  Per-layer KV cache for one sequence:
    KV_bytes_per_layer = 2 × S × H_kv × d_head × bytes_per_element

    2          = K tensor + V tensor
    S          = context length (number of tokens)
    H_kv       = KV heads (GQA: 8 for LLaMA-3 70B; MHA: 64)
    d_head     = per-head dimension (128 for LLaMA-3 70B)
    bytes      = 2 (BF16) or 1 (INT8/FP8) per element

  Total KV cache for one sequence:
    KV_total = KV_bytes_per_layer × L  (L = number of transformer layers)

KV CACHE SIZING — LLAMA-3 70B GQA (H_kv=8, d_head=128, L=80, BF16):

  Per-layer: 2 × S × 8 × 128 × 2 bytes = 4,096 × S bytes
  80 layers: 80 × 4,096 × S = 327,680 × S bytes = 320 KB × S

  Context      KV Cache/seq   H100 headroom*  Concurrent seqs*
  ──────────────────────────────────────────────────────────────
  S=1,024      0.34 GB        ~24 GB free      70 seqs
  S=2,048      0.67 GB        ~24 GB free      35 seqs
  S=4,096      1.34 GB        ~24 GB free      17 seqs
  S=8,192      2.68 GB        ~24 GB free       8 seqs
  S=32,768     10.74 GB       ~24 GB free       2 seqs
  S=128,000    41.9 GB        ~24 GB free       0 (barely fits alone!)
  ──────────────────────────────────────────────────────────────
  * H100 free memory = 80 GB - 140 GB weights = NEGATIVE (model doesn't fit!)
  * Assumes H200 (141 GB): free = 141 - 140 = 1 GB (very tight for BF16 weights)
  * Practical long-context: use FP8 weights (70 GB) -> 141-70=71 GB for KV

  KV CACHE CAPACITY vs PLATFORM:
    H100 (80 GB):   0 GB free for 70B BF16 weights alone. Need TP=2 minimum.
    H200 (141 GB):  ~1 GB for BF16 weights. Use FP8 weights (70 GB) = 71 GB KV.
```

```
    MI300X (192 GB): 192-140=52 GB free. Fits S=128K (needs 41.9 GB) comfortably.
    B200 (192 GB):  Same as MI300X capacity.
```

4.9.2 KV Cache Optimization Hierarchy

```
KV CACHE OPTIMIZATION OPTIONS (ordered by impact and ease):

  1. GQA (Grouped Query Attention): 8× KV reduction vs MHA
     Quality cost: ~0% (well-tuned GQA matches MHA quality)
     Implementation: architectural choice — use at model training time
     Already in: all modern models (LLaMA-3, Mistral, Gemma)

  2. INT8 KV cache: 2× more cache for same memory footprint
     Quality cost: ~0.1-0.5% on most benchmarks
     Implementation: apply per-channel quantization to K, V tensors
     Supported in: vLLM (--kv-cache-dtype fp8_e5m2), TRT-LLM
     Effect at LLaMA-3 70B H200 (71 GB KV budget):
       BF16: 41.9 GB for S=128K -> fits 1 seq
       INT8: 20.9 GB for S=128K -> fits 3 concurrent seqs

  3. Prefix caching (shared system prompt):
     Many LLM deployments have identical system prompts across requests.
     Cache the KV for the system prompt once, reuse for all requests.
     vLLM: prefix caching enabled by default (--enable-prefix-caching)
     Effect: 50-80% KV reduction for typical chatbot workloads
     Implementation: radix tree for KV block sharing (Chapter 11)

  4. KV eviction (sliding window / streaming LLM):
     Evict KV entries beyond a fixed window length.
     Retains only the last W tokens' KV cache.
     Effect: O(1) memory regardless of context length
     Quality cost: significant for long-range reasoning tasks
     Use: chatbots, not summarization or multi-doc reasoning

  5. KV tiering (GPU -> CPU -> NVMe):
     Hot KV (recent tokens): keep in GPU HBM
     Warm KV (older tokens): offload to CPU DRAM via NVLink/PCIe
     Cold KV (very old tokens): offload to NVMe (high latency, avoid)
     Requires: prefetch pipeline to bring cold KV warm before needed
     Effect: 5-10× more effective context per GPU HBM dollar

  6. MQA (Multi-Query Attention):
     H_kv = 1 instead of 8 (GQA) or 64 (MHA)
```

```
    KV reduction: 8× vs GQA, 64× vs MHA
    Quality cost: ~1-2% degradation vs GQA on standard benchmarks
    Use: extreme memory pressure only (very long context, many users)
```

4.10 Profiling Memory Bottlenecks

4.10.1 Nsight Compute Memory Profiling

```
KEY NSIGHT COMPUTE MEMORY COUNTERS:

  Counter                                Target     Low Signal Interpretation
  ──────────────────────────────────────────────────────────────────────────
  l1tex_t_bytes_pipe_lsu_mem_global_    Minimize   High = too much HBM traffic
  _op_ld.sum (global memory loads)

  l2_global_load_bytes                 Minimize   High = L1 not catching data

  smsp__l1tex_t_sector_hit_rate.avg     >50%       Low = poor SMEM usage

  l1tex_t_sector_hit_rate (L1 hit rate) >70%       Low = no data reuse in L1

  smsp__warp_issue_stalled_long_        Minimize   High = kernel waiting on HBM
  scoreboard_per_warp_active.avg

  sm__sass_l1tex_t_sectors_pipe_lsu_   Minimize   >32x ratio = bad coalescing
  mem_global_op_ld.sum /
  sm__sass_l1tex_t_requests_pipe_lsu_
  mem_global_op_ld.sum
  (sectors per request ratio)
  ──────────────────────────────────────────────────────────────────────────


NSIGHT COMPUTE ROOFLINE TAB:
  Opens a visual roofline plot for the selected kernel.
  Your kernel appears as a dot.
  Dot on memory roof slope -> memory-bound.
  Dot at compute ceiling -> compute-bound.
  Dot below both -> launch/occupancy bound.

COMMAND LINE PROFILING:
  # Full memory analysis for a specific kernel
  ncu --set full \
```

```
      --kernel-name "flash_fwd_splitkv_kernel" \
      --metrics l1tex_t_bytes_pipe_lsu_mem_global_op_ld.sum,\
                l2_global_load_bytes,\
                smsp__l1tex_t_sector_hit_rate.avg \
      python inference.py

  # Quick HBM bandwidth check
  ncu --metrics gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed \
      python inference.py
```

4.10.2 AMD ROCm Memory Profiling (MI300X)

```
AMD OMNIPERF + ROCPROF MEMORY ANALYSIS:

  # Omniperf roofline analysis
  omniperf profile --name workload_run -- python train.py
  omniperf analyze --path workload_run/ --roof-only

  # rocprof memory counters
  rocprof --hsa-trace --stats -i counters.txt python infer.py

  Key MI300X counters (counters.txt):
    SQ_INSTS_VMEM_RD    — HBM read instructions
    SQ_INSTS_VMEM_WR    — HBM write instructions
    L2_HITS             — L2 cache hits (Infinity Cache = 32 MB L3)
    L2_MISSES           — L2 misses -> HBM fetches
    FETCH_SIZE          — bytes fetched from memory
    WRITE_SIZE          — bytes written to memory

  MI300X MEMORY ARCHITECTURE DIFFERENCE vs H100:
    H100: 50 MB L2 cache (standard SRAM)
    MI300X: 32 MB Infinity Cache (L3, per-MCD: 4 MCDs × 8 MB)
    The Infinity Cache is explicitly in the critical path between
    XCDs (compute) and HBM controllers (MCDs).
    Higher effective bandwidth for working sets < 32 MB.
    Cache miss -> HBM: same 5.3 TB/s as H100's 3.35 TB/s (58% more).
```

4.11 The Memory Performance Decision Tree

```
MEMORY OPTIMIZATION DECISION TREE:
```

```
START: Kernel or operation to optimize
        |
        v
STEP 1: Measure Arithmetic Intensity
        (Nsight Compute Roofline tab or manual: FLOPs / HBM_bytes)
        |
        v
┌──────────────────────────────────────────────────────────────┐
|  Is AI < Ridge Point (~295 on H100)?                         |
└──────┬──────────────────────────────┬──────────────────────┘
     | YES                          | NO
     v                              v
MEMORY-BOUND                  COMPUTE-BOUND
Apply in order:               Apply in order:
1. Operator fusion            1. FP8 quantization (2× TFLOPS)
   (biggest win, always)      2. Fix TC alignment (shapes ÷ 16)
2. Weight quantization        3. Increase batch size
   (INT4/FP8)                 4. Optimize tile sizes / CUTLASS config
3. Fix access coalescing      5. Reduce register pressure
4. Increase SMEM tile size    6. FP4 (Blackwell only)
5. Async prefetch (double     7. Operator fusion to reduce
   buffer SMEM tiles)            launch overhead
6. KV cache compression
7. Increase batch size
   (shift regime upward)

ALSO CHECK: Is SM utilization < 40% during active computation?
  YES -> overhead-bound (not memory or compute)
  Apply:
    1. CUDA Graphs (capture CPU dispatch overhead)
    2. torch.compile (eliminate Python overhead)
    3. Fuse small operations manually
    4. C++ runtime (eliminate Python interpreter)
    5. Increase batch size (amortize overhead across more work)

VALIDATE: Always check system MFU before and after.
  Kernel improvement that doesn't improve system MFU = exposed next bottleneck.
  Keep drilling down until MFU improves end-to-end.
```

4.12 Key Takeaways and Review Questions

```
HARDWARE NUMBERS TO MEMORIZE COLD — H100 SXM5: [SHIPPED]
  Registers:    65,536 × 32-bit per SM  (~1 cycle)
  L1/SMEM:      228 KB per SM, configurable  (~20-30 cycles)
  L2 Cache:     50 MB total, shared  (~150 cycles)
  HBM3:         3.35 TB/s bandwidth · 80 GB capacity  (~400+ cycles)
  PCIe Gen5:    128 GB/s  (~1 μs)
  NVLink 4.0:   900 GB/s per GPU  (<1 μs intra-node)
  Ridge point:  ~295 FLOP/byte (BF16, dense)

KEY PRINCIPLES:

  1. DECODE IS ALWAYS MEMORY-BANDWIDTH-BOUND:
     At batch=1, AI ≈ 2 FLOP/byte for all LLM operations.
     Far below any GPU's ridge point. More TFLOPS = zero improvement.
     Optimize: HBM bandwidth, KV cache size, weight quantization.

  2. FUSION IS THE HIGHEST-LEVERAGE OPTIMIZATION:
     Un-fused RMSNorm at LLaMA-3 70B: 21 GB avoidable HBM per step.
     Fused: 0 GB wasted. 6-8% step time improvement, no algorithm change.
     Rule: every elementwise op after a GEMM should be fused.

  3. HBM CAPACITY GROWS SLOWER THAN BANDWIDTH:
     BW: 13× from V100 → B300. Capacity: ~3.5× same period.
     Each generation: quantization covers more of the capacity gap.
     INT4 70B (35 GB) fits on A100-class hardware; BF16 needs H100-class.

  4. COALESCING IS NON-NEGOTIABLE:
     Uncoalesced vs sequential access: 13× bandwidth difference on same GPU.
     Always stage scattered access through SMEM with coalesced tile loads.
     Check: Nsight Compute sectors-per-request ratio should be ~1.

  5. FLASHATTENTION ELIMINATED THE ATTENTION BOTTLENECK:
     Standard attention: O(S²) HBM traffic.
     FlashAttention: O(S) HBM traffic (5-10× reduction at S=4K).
     At S=128K: standard attention is physically impossible on any existing GPU.
     FlashAttention is not optional — it is a prerequisite for long context.

  6. THE H200 LESSON (BANDWIDTH > TFLOPS FOR INFERENCE):
     Same Tensor Cores as H100. +43% HBM bandwidth. +76% HBM capacity.
     Result: ~40% more 70B inference throughput. Zero compute improvement.
     The message: for LLM serving, buy memory bandwidth, not compute.
```

```
DECODE THROUGHPUT FORMULA (memory-bound, batch=1):
  tokens/sec ≈ HBM_bandwidth_GB/s / model_size_GB × 0.70
  H100 BF16:   3350 / 140 × 0.70 = ~17 tok/s  (LLaMA-3 70B)
  H200 BF16:   4800 / 140 × 0.70 = ~24 tok/s  (single GPU — fits!)
  MI300X BF16: 5300 / 140 × 0.70 = ~27 tok/s  (single GPU — fits!)
  B200 BF16:   8000 / 140 × 0.70 = ~40 tok/s  (single GPU — fits!)

KV CACHE FORMULA (LLaMA-3 70B GQA, BF16):
  KV_GB = 2 × S × H_kv × d_head × L × 2B / 1e9
        = 2 × S × 8 × 128 × 80 × 2 / 1e9
        = S × 3.28 × 10^-4 GB
  At S=4,096:   1.34 GB per sequence
  At S=32,768:  10.7 GB per sequence
  At S=128,000: 41.9 GB per sequence
```

Review Questions

Difficulty: Foundational

Question 1 (Memory Wall and Decode): A new H200 server has 4.8 TB/s HBM bandwidth and 141 GB capacity. (a) Calculate the ridge point (BF16, peak 989 TFLOPS). (b) For LLaMA-3 70B BF16 at batch=1, are the FFN GEMMs compute-bound or memory-bound? Prove it with arithmetic intensity calculation. (c) An engineer proposes upgrading H200 to a hypothetical 1,500 TFLOPS variant to improve 70B decode throughput. Is this a good idea? Justify with numbers.

Expected answer: (a) Ridge = 989 / 4.8 = 206 FLOP/byte. H200 has a LOWER ridge point than H100 (206 vs 295) because bandwidth grew faster than compute. (b) At batch=1: FFN GEMM shape [1 × 8192] × [8192 × 28672]. FLOPs = 2 × 1 × 8192 × 28672 = 470M. Bytes loaded = 8192 × 28672 × 2 (BF16) = 470 MB (weights) + tiny activations. AI ≈ 1 FLOP/byte. Far below 206 FLOP/byte ridge. Memory-bound. (c) Bad idea. At batch=1, decode is memory-bandwidth-bound (AI ≈ 1). Increasing TFLOPS from 989 → 1500 has zero effect on throughput — the GPU is waiting for HBM data, not compute cycles. The correct intervention is: increase HBM bandwidth (buy B200), apply weight quantization (INT4 → 4× throughput), or increase batch size (raise AI toward ridge point). Never recommend TFLOPS for memory-bound workloads.

Difficulty: Intermediate

Question 2 (FlashAttention vs Standard Attention): You are comparing standard attention to FlashAttention for LLaMA-3 70B on H100 at S=8,192, d_head=128, 64 heads, BF16. (a) Calculate HBM traffic for standard attention (one layer, all heads). (b) Calculate HBM traffic for

FlashAttention (same configuration). (c) If the H100 HBM bandwidth is 3.35 TB/s, what is the minimum possible attention execution time for each approach (ignoring compute)?

Expected answer: (a) Standard: per head per layer = $4 \times S^2 \times 4B + 4 \times S \times d \times 2B = 4 \times 8192^2 \times 4 + 4 \times 8192 \times 128 \times 2$ = 1,073 MB + 8 MB = 1,081 MB per head. 64 heads = 69 GB per layer. 80 layers = 5,530 GB total. (b) FlashAttention: loads Q, K, V each once, writes O once = $4 \times S \times d \times 2B$ per head = $4 \times 8192 \times 128 \times 2$ = 8 MB per head. 64 heads = 528 MB per layer. 80 layers = 42 GB total. (c) Standard: 5,530 GB / 3,350 GB/s = 1.65 seconds minimum. FlashAttention: 42 GB / 3,350 GB/s = 12.5 ms minimum. Speedup: 132× from HBM traffic reduction alone. At S=8K, standard attention is not merely slow — it is 1.65 seconds per forward pass on attention traffic alone, making any production use impossible.

Difficulty: Principal Level

Question 3 (KV Cache Platform Selection): A customer requires serving LLaMA-3 70B at 128K context with 8 concurrent users. All hardware uses BF16 weights. Design the minimum hardware deployment. Show the memory math for: (a) H100 SXM5, (b) H200 SXM5, (c) MI300X. For the viable options, calculate how INT8 KV cache changes concurrent user capacity.

Expected answer: KV per sequence at S=128K: $2 \times 128000 \times 8 \times 128 \times 80 \times 2B$ = 41.9 GB. 8 users: 8 × 41.9 = 335 GB KV cache needed + 140 GB weights = 475 GB total. (a) H100 (80 GB): model doesn't fit on a single GPU (140 > 80). Even TP=2 = 160 GB — no room for KV. Need TP=8 (8 GPUs) with 640 GB total. KV budget = 640 - 140 = 500 GB. Fits 8 users (335 GB). But extremely expensive — 8 GPUs for one serving instance. (b) H200 (141 GB): 140 GB weights fit on 2 GPUs (TP=2 = 282 GB). KV budget = 282 - 140 = 142 GB. 142 / 41.9 = 3 concurrent users at 128K. Need 3 TP=2 pairs (6 GPUs) for 8 users. With INT8 KV: 41.9/2 = 20.95 GB per user. 142 / 20.95 = 6 users on 2 GPUs. 2 TP=2 pairs (4 GPUs) for 8 users. (c) MI300X (192 GB): 140 GB weights fit on 1 GPU. KV budget = 192 - 140 = 52 GB. 52 / 41.9 = 1.24 users (barely 1). Need 8 GPUs minimum (each serving 1 user independently). With INT8 KV: 52 / 20.95 = 2.5 users per GPU → 4 MI300X for 8 users. With FP8 weights (70 GB): 192 - 70 = 122 GB KV. 122 / 20.95 = 5.8 → 2 MI300X GPUs (each handles ~4 users). MI300X + FP8 weights + INT8 KV = minimum deployment.

Difficulty: Principal Level

Question 4 (Fusion ROI Analysis): You profile a LLaMA-3 70B training step and find: step time = 120 ms, MFU = 38%. Nsight Systems shows 160 un-fused RMSNorm kernels occupying 8.2 ms of wall time. A colleague proposes implementing kernel fusion as a 3-week engineering project. (a) What is the maximum possible MFU improvement from fusing all RMSNorm ops? (b) Is 3 weeks of engineering time worth it? Calculate the threshold ROI. (c) After implementing fusion, MFU goes from 38% to 41% — not the theoretical maximum. Name three reasons the full HBM savings might not translate 1:1 to MFU.

Expected answer: (a) 8.2 ms saved / 120 ms total = 6.8% step time reduction. New step time = 111.8 ms. New MFU = 38% × (120/111.8) = 40.8%. Maximum improvement ≈ 2.8 percentage points on MFU. (b) Depends on cluster scale and training duration. At 1024 H100s for 30 days: 6.8% step time reduction = 6.8% compute saved = 6.8% × 1024 × 30 × 24 = 49,766 GPU-hours saved. At $3/GPU-hr: $149,000 saved. 3 weeks of 1 engineer = ~$30,000 cost. ROI = 5×. Worth doing. (c) Three reasons fusion doesn't achieve full HBM savings in MFU terms: (1) Exposing the next bottleneck: fusing norms may reveal AllReduce communication as the new dominant cost — the step time improves less than expected because communication was already partially overlapped. (2) New register pressure: fused kernels use more registers (holding norm statistics and GEMM tile simultaneously), potentially reducing occupancy and partially offsetting the bandwidth gain. (3) L2 cache behavior change: un-fused kernels may have been warming the L2 cache for subsequent operations — fused kernels change the cache footprint, potentially increasing L2 misses elsewhere.

Further Reading

1. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* — Dao et al. (2022). `arxiv.org/abs/2205.14135` The IO complexity proof is the best applied example of roofline-based algorithm design in the literature. Read the formal proof in Section 3.
2. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning* — Dao (2023). `arxiv.org/abs/2307.08691` Extends FA to H100 WGMMA and improves parallelism for longer sequences.
3. *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision* — Shah et al. (2024). `arxiv.org/abs/2407.08608` H100-specific WGMMA async overlap. The state of the art for attention throughput.
4. *JEDEC HBM3 Specification* (JESD238). `jedec.org` The primary source for HBM3 physical parameters: bus width, data rate, timing.
5. *NVIDIA H100 Tensor Core GPU Architecture Whitepaper* (2022). Section 4: Memory Architecture. Full HBM3 bandwidth derivation and L2 design. `resources.nvidia.com/en-us-tensor-core`
6. *Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking* — Jia et al. (2018). `arxiv.org/abs/1804.06826` Best detailed analysis of memory hierarchy characteristics from measurement.

— End of Chapter 4 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved